@code.withnarayan

C++ Programming

Basic to Advanced

Notes

Introduction to C++

1 What is C++?

- C++ is a general purpose, high-level, object-oriented programming language.
- It was developed by Bjarne Stroustrup
- in 1983 at Bell Labs.
- C++ is an extension of C language and supports both procedural and object-oriented programming.
- It is widely used in system programming, game development, embedded systems, etc.

2 History of C++

- Initially called "C with Classes".
- Developed by Bjarne Stroustrup in 1983.
- First commercial version released in 1985.
- Standardized by ISO (International Organization for Standardization).
- Latest major standard: C++23 (2023).

Fun Fact: C++ was designed to add object-oriented features to the C language.

3 Key Features of C++

- Simple and powerful language.
- Supports both procedural and OOP.
- Fast and efficient (close to hardware).
- Portable (write once, run anywhere).
- Rich standard library (STL).
- Supports function overloading and operator overloading.
- Dynamic memory management.
- Used for system and application development.
- Large community support.

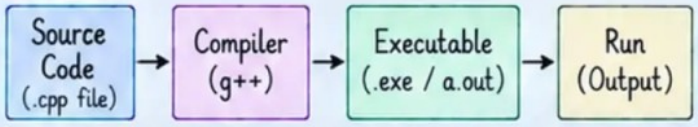
4 C++ vs C

Feature	C	C++
Paradigm	Procedural	Procedural + OOP
Classes & Objects	Not available	Available
Function Overloading	Not available	Available
Operator Overloading	Not available	Available
Namespaces	Not available	Available
Standard Library	Limited	Rich (STL)
Exception Handling	Not available	Available
I/O	printf, scanf	cin, cout
Memory Management	malloc, free	new, delete
Use Case	System programming	Large applications

5 How C++ Works?

- C++ code is written in a text file with .cpp extension.
- The code is compiled using a C++ compiler (e.g. g++).
- Compilation converts the code into machine code (executable file).
- The executable file is then run to get the output.

Process:



6 Applications of C++

- Operating Systems (e.g. Windows, Linux parts)
- Game Development (e.g. Unreal Engine)
- Web Browsers (e.g. Chrome, Firefox)
- Database Management Systems
- Embedded Systems
- Compilers and Interpreters
- Real-time Systems (e.g. Air traffic control)
- Scientific and Simulation Software
- Finance and Trading Systems
- High Performance Applications

7 Example: First C++ Program

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello, C++!" << endl;
    return 0;
}
```

Output:
Hello, C++!

Note:

- C++ is case sensitive.
- Every statement ends with a semicolon (;).
- main() is the starting point of the program.

C++ Program Structure + First Program

1 Structure of a C++ Program

- A C++ program is written in a specific structure.
- It consists of header files, namespace, main function and statements.
- All statements end with a semicolon (;).

```
#include <header_file>     // header file(s)
using namespace std;       // standard namespace

int main()                 // main function (entry point)
{
    // variable declaration
    // statements
    // logic
    return 0;              // indicate successful execution
}
```

2 Explanation of Each Part

Part	Description
#include <...>	Includes required header files.
using namespace std;	Allows us to use standard library names (like cout, cin) without prefixing std::
int main()	Main function (starting point of every C++ program).
{ }	Block that contains the program statements.
return 0;	Indicates successful execution of the program to the operating system.
; (semicolon)	Ends a statement.
// ...	Single line comment (ignored by compiler).
/* ... */	Multi-line comment (ignored by compiler).

3 Header Files

- Header files contain declarations of functions, classes and other components.
- We include them using #include.
- Common header files:

Header File	Purpose
iostream	Input and output (cin, cout)
cmath	Mathematical functions
string	String class
vector	Vector container
algorithm	Algorithms (sort, etc.)

4 Namespace in C++

- The standard library elements (like cout, cin) are defined inside the namespace std.
- Instead of writing std::cout, we can use using namespace std; to access them directly.

Note:

- Using namespace std; is convenient but can cause name conflicts in large programs.
- In professional projects, it is often recommended to use std:: (e.g. std::cout).

5 The main() Function

- Execution of a C++ program always starts from main().
- It must return an integer value.
- int main() is the most common form.
- It can also be written as int main(int argc, char* argv[]) to accept command line arguments.

6 First C++ Program

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello, C++!" << endl;
    return 0;
}
```

Output:

Hello, C++!

Note:

- cout is used to print output.
- endl moves the cursor to the next line.
- return 0; indicates successful execution of the program.
- This is a simple C++ program that prints a message on the screen.

Variables + Data Types + Constants

1 Variables in C++

- A variable is a named memory location used to store data.
- Its value can be changed during program execution.
- In C++, every variable must be declared with a data type.

Syntax:

```
data_type variable_name;
data_type variable_name = value;
```

2 Rules for Variable Names

- Can contain letters (a-z, A-Z), digits (0-9) and underscore (_).
- Must start with a letter or underscore.
- Cannot start with a digit.
- No spaces are allowed.
- Case sensitive (age and Age are different).
- Cannot use C++ keywords (int, float, class, etc.) as variable names.

Examples:

Valid: age, _count, totalMarks, num1
Invalid: 1age, total marks, int, floatValue%

3 Data Types in C++

Type	Size (bytes)	Range (approx.)	Description
int	4	-2^{31} to $2^{31}-1$	Stores integers
float	4	3.4×10^{-38} to 3.4×10^{38}	Stores decimal numbers
double	8	1.7×10^{-308} to 1.7×10^{308}	Stores larger decimal numbers
char	1	-128 to 127	Stores single character
bool	1	true or false	Stores boolean values
short int	2	-2^{15} to $2^{15}-1$	Smaller integer range
long int	8	-2^{63} to $2^{63}-1$	Larger integer range
long long int	8	-2^{63} to $2^{63}-1$	Very large integer range
string	–	–	Stores sequence of characters (defined in <string> header)

4 Examples of Variable Declaration

```
int: age = 20;
float height = 5.9;
double salary = 45000.75;
char grade = 'A';
bool isStudent = true;
string name = "Narayan";
```

Output:

```
age = 20
height = 5.9
salary = 45000.75
grade = A
isStudent = 1
name = Narayan
```

5 Constants in C++

- A constant is a fixed value that cannot be changed during program execution.
- Constants are used to prevent accidental modification of important values.
- We can define constants using const keyword or #define preprocessor directive.

Syntax:

```
const data_type constant_name = value;
#define CONSTANT_NAME value
```

6 Examples of Constants

```
const int MAX = 100;
const float PI = 3.14;
#define BRAND "C++"
#define SIZE 50
```

Output:

```
MAX = 100
PI = 3.14
BRAND = C++
SIZE = 50
```

Note:

- const is preferred in modern C++.
- Use UPPERCASE for #define constants.
- Constants cannot be modified once defined.

Input / Output (cin, cout) + Operators

1 Input and Output in C++

- C++ provides **cout** for output (display) and cin for input (read).
- Both are defined in <iostream> header file.
- We use namespace std; to avoid writing std:: again and again.

Syntax:

```
#include <iostream>
using namespace std;

cout << "output";      // for output
cin >> variable;       // for input
```

2 Examples of cin and cout

```
#include <iostream>
using namespace std;
int main()
{
    int age;
    string name;
    cout << "Enter your name: ";
    cin >> name;
    cout << "Enter your age: ";
    cin >> age;
    cout << "Hello " << name << "!\n";
    cout << "You are " << age << " years old.";
    return 0;
}
```

Output:

```
Enter your name: Narayan
Enter your age: 20
Hello Narayan!
You are 20 years old.
```

3 Some Important I/O Points

- Use **cin >>** for single word input.
- Use getline(cin, **str**) for reading a full line (with spaces).
- cout << can be chained using <<.
- We can use manipulators like **endl** to move to next line.
- cin ignores whitespace (space, tab, newline).
- For multiple inputs: cin >> a >> b >> c;

Example: Using getline()

```
#include <iostream>
using namespace std;
int main()
{
    string name;
    cout << "Enter full name: ";
    getline(cin, name);
    cout << "Name: " << name;
    return 0;
}
```

Output:

```
Enter full name:
Narayan Prasad
Name: Narayan Prasad
```

4 Types of Operators in C++

Type	Operators
Arithmetic	+ - * / %
Relational	== != > < >= <=
Logical	&& || !
Assignment	= += -= *= /= %=
Increment / Decrement	++ --
Bitwise	& | ^ ~ << >>
Conditional (Ternary)	?:
Special	sizeof , (comma)

5 Arithmetic Operators

Operator	Meaning	Example	Result
+	Addition	a + b	7 (3+4)
-	Subtraction	a - b	-1 (3-4)
*	Multiplication	a * b	12 (3*4)
/	Division	a / b	0 (3/4)
%	Modulus (Remainder)	a % b	3 (7%4)

6 Relational Operators

Operator	Meaning	Example (a=3, b+4)	Result
==	Equal to	a == b	false
!=	Not equal to	a != b	true
>	Greater than	a > b	false
<	Less than	a < b	true
>=	Greater than or equal	a >= b	false
<=	Less than or equal	a <= b	true

7 Logical Operators

Operator	Meaning	Example (a=3, b+4)	Result
&&	Logical AND	(a < 5 && b < 5)	true
||	Logical OR	(a < 5 || b < 2)	true
!	Logical NOT	!(a < 5)	false

8 Increment / Decrement Operators

Operator	Meaning	Example	Result (a=5)
++	Increment by 1	++a	6
--	Decrement by 1	--a	4
a++	Post-increment	a++	(5, then 6)
a--	Post-decrement	a--	(5, then 4)

Decision Making (if, if-else, switch)

1 What is Decision Making?

- Decision making allows a program to execute different blocks of code based on a condition.
- It is used to control the flow of execution.
- C++ provides if, if-else, else-if ladder and switch statement for decision making.

2 if Statement

- The if statement is used to execute a block of code only if the condition is true.
- If the condition is false, the block is skipped.

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int age = 18;
    if(age >= 18)
    {
        cout << "You are an adult";
    }
    return 0;
}
```

Output:

```
You are an adult
```

3 if-else Statement

- The if-else statement executes one block if the condition is true, otherwise it executes another block.

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if(num % 2 == 0)
    {
        cout << "Even Number";
    }
    else
    {
        cout << "Odd Number";
    }
    return 0;
}
```

Output:

```
Enter a number: 5
Odd Number
```

4 else-if Ladder

- Used when we have multiple conditions to check.
- It checks each condition one by one.

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int marks;
    cout << "Enter marks: ";
    cin >> marks;
    if(marks >= 90)
        cout << "Grade: A";
    else if(marks >= 75)
        cout << "Grade: B";
    else if(marks >= 50)
        cout << "Grade: C";
    else
        cout << "Grade: D";
    return 0;
}
```

Output:

```
Enter marks: 82
Grade: B
```

5 switch Statement

- The switch statement is used to execute one block of code from multiple options based on the value of an expression.
- It is an alternative to multiple if-else conditions.

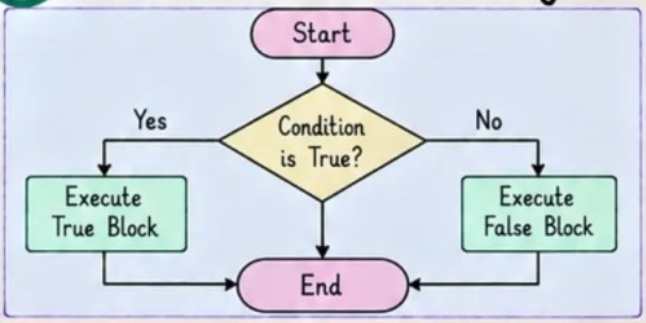
Example:

```
#include <iostream>
using namespace std;
int main()
{
    int choice;
    cout << "Enter choice (1-3): ";
    cin >> choice;
    switch(choice)
    {
        case 1: cout << "Option 1"; break;
        case 2: cout << "Option 2"; break;
        case 3: cout << "Option 3"; break;
        default: cout << "Invalid choice";
    }
    return 0;
}
```

Output:

```
Enter choice (1-3): 2
Option 2
```

6 Flowchart of Decision Making



Key Points:

- Use if when only one condition is needed.
- Use if-else when there are two possible outcomes.
- Use else-if ladder when multiple conditions are there.
- Use switch when checking a single expression against multiple constant values (int, char, enum).
- Always use break in switch to avoid fall-through.

Loops (for, while, do-while)

1 What are Loops?

- Loops are used to execute a block of code repeatedly until a certain condition is met.
- They help to reduce code repetition.
- C++ provides three types of loops:
  1. for loop
  2. while loop
  3. do-while loop

Example: Print numbers 1 to 5

```cpp
#include <iostream>
using namespace std;
int main()
{
    for(int i = 1; i <= 5; i++)
    {
        cout << i << " ";
    }
    return 0;
}
```

Output:

```
1 2 3 4 5
```

2 for Loop

- Used when the number of iterations is known.
- Consists of initialization, condition and update expression.
- Executed in the order: init → condition → body → update.

Syntax:

```cpp
for(initialization; condition; update)
{
    // statements
}
```

Example: Calculate sum of first n numbers

```cpp
#include <iostream>
using namespace std;
int main()
{
    int n, sum = 0;
    cout << "Enter n: ";
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        sum += i;
    }
    cout << "Sum = " << sum;
    return 0;
}
```

Output:

```
Enter n: 5
Sum = 15
```

3 while Loop

- Used when the number of iterations is not known in advance.
- The condition is checked before executing the loop body.
- If the condition is false, the loop body is not executed even once.

Syntax:

```cpp
while(condition)
{
    // statements
}
```

Example: Print numbers 1 to 5

```cpp
#include <iostream>
using namespace std;
int main()
{
    int i = 1;
    while(i <= 5)
    {
        cout << i << " ";
        i++;
    }
    return 0;
}
```

Output:

```
1 2 3 4 5
```

4 do-while Loop

- Similar to while loop, but the condition is checked after executing the loop body.
- So, the loop body is executed at least once.

Syntax:

```cpp
do
{
    // statements
} while(condition);
```

Example: Print numbers 1 to 5

```cpp
#include <iostream>
using namespace std;
int main()
{
    int i = 1;
    do
    {
        cout << i << " ";
        i++;
    } while(i <= 5);
    return 0;
}
```

Output:

```
1 2 3 4 5
```

5 Comparison of Loops

Loop	Condition Check	Execution Count	Use Case
for	Before	0 or more	When iterations are known
while	Before	0 or more	When iterations are not known
do-while	After	1 or more	When at least one

6 Key Points

- Use for loop when the number of iterations is fixed.
- Use while loop when the number of iterations is not known.
- Use do-while loop when the code must execute at least once.
- Make sure to update the loop variable to avoid infinite loop.
- Loops are widely used in problems like arrays, patterns,

Functions + Function Overloading

1 What is a Function?

- A function is a block of code that performs a specific task.
- It helps to organize code, reuse it and make the program modular.
- In C++, a function is defined using a return type, function name, parameters (optional) and a body.

Syntax:

```
return_type function_name(parameters)
{
    // statements
    return value;   // (if return type is not void)
}
```

2 Types of Functions

- Library functions - Provided by C++ (e.g, cout, cin, sqrt()).
- User-defined functions - Created by the programmer.
- Function with arguments and return value.
- Function with arguments and no return value.
- Function with no arguments and return value.
- Function with no arguments and no return value.

Example:

```
#include <iostream>
using namespace std;
int add(int a, int b)
{
    return a + b;
}
int main()
{
    int x = 5, y = 3;
    cout << "Sum = " << add(x, y);
    return 0;
}
```

Output:

```
Sum = 8
```

3 Function with No Arguments

Example:

```
#include <iostream>
using namespace std;
void greet()
{
    cout << "Hello, C++!";
}
int main()
{
    greet();
    return 0;
}
```

Output:

```
Hello, C++!
```

4 Function with Arguments (No Return)

Example:

```
#include <iostream>
using namespace std;
void greet(string name)
{
    cout << "Hello, " << name;
}
int main()
{
    greet("Narayan");
    return 0;
}
```

Output:

```
Hello, Narayan
```

5 Function with Return Value

Example:

```
#include <iostream>
using namespace std;
int square(int n)
{
    return n * n;
}
int main()
{
    int num = 4;
    cout << "Square = " << square(num);
    return 0;
}
```

Output:

```
Square = 16
```

6 Function Overloading

- Function overloading means multiple functions with the same name but different parameters.
- It is a form of compile-time polymorphism.
- Functions must differ in the number or type of parameters.

Example:

```
#include <iostream>
using namespace std;
int add(int a, int b)
{   return a + b; }
double add(double a, double b)
{   return a + b; }
int main()
{
    cout << add(2, 3) << endl;     // calls int version
    cout << add(2.5, 3.5);         // calls double version
    return 0;
```

Output:

```
5
6
```

7 Key Points

- Functions reduce code repetition.
- Always declare a function before using it.
- Function overloading is based on number or type of parameters (not return type).

Arrays + Strings

1 Arrays in C++

- An array is a collection of elements of the same data type stored in contiguous memory locations.
- Elements are accessed using index (starts from 0).
- Array size must be specified at the time of declaration.
- Can store multiple values of the same type.

Syntax:

```
data_type array_name[size];
```

2 Example: Declare and Initialize array

```
#include <iostream>
using namespace std;
int main()
{
    int arr[5] = {10, 20, 30, 40, 50};
    cout << "Elements are: ";
    for(int i = 0; i < 5; i++)
    {
        cout << arr[i] << " ";
    }
    return 0;
}
```

Output:

```
Elements are:
10 20 30 40 50
```

3 Accessing Array Elements

- Use index to access individual elements.
- Index starts from 0 and goes up to size-1.
- We can also modify elements using index.

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int arr[3] = {5, 10, 15};
    cout << "First: " << arr[0] << endl;
    cout << "Second: " << arr[1] << endl;
    arr[1] = 20;      // modify element
    cout << "Updated Second: " << arr[1];
    return 0;
}
```

Output:

```
First: 5
Second: 10
Updated Second: 20
```

4 Multi-dimensional Arrays

- Used to store data in table form (matrix).

Example: 2D Array

```
int mat[2][3] = {{1, 2, 3}, {4, 5, 6}};
cout << mat[0][1];   // Output: 2
```

Memory (2 x 3)

1	2	3
4	5	6

mat[2][3]

5 Strings in C++

- A string is a sequence of characters.
- In C++, we can use character array or the string class (from <string> library).
- string class is easier and more powerful than character arrays.

6 Using Character Array (C-style string)

- A character array is a string ending with '\0' (null character).
- Can be declared and initialized like an array.

Example:

```
#include <iostream>
using namespace std;
int main()
{
    char str[] = "Hello";
    cout << "String: " << str;
    return 0;
}
```

Output:

```
String: Hello
```

Memory

H	e	l	l	o	\0
0	1	2	3	4	5

7 Using string Class (Recommended)

- Provided by <string> header file.
- No need to worry about size or null character.
- Supports many useful functions.

Example:

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string name = "Narayan";
    cout << "Hello " << name;
    return 0;
}
```

Output:

```
Hello Narayan
```

8 Some Useful string Functions

Function	Description	Example
length()	Returns length of string	str.length()
push_back()	Adds a character at end	str.push_back('a')
append()	Adds a string at end	str.append("XYZ")
substr()	Returns a substring	str.substr(0, 3)
find()	Finds position of substring	str.find("a")
compare()	Compares two strings	str1.compare(str2)

9 Key Points

- Arrays store multiple values of same type.
- Array index starts from 0.
- Use string class instead of character array (recommended).

Pointers + References

1 What is a Pointer?

- A pointer is a variable that stores the memory address of another variable.
- It points to a memory location.
- In C++, we use * (dereference) and & (address of) operators with pointers.
- Pointer helps in dynamic memory allocation, array handling, function arguments, etc.

2 Declaration and Initialization

Syntax:

```
data_type  *pointer_name;
pointer_name = &variable_name;
```

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int a = 10;
    int *ptr = &a;

    cout << "Value of a: " << a << endl;
    cout << "Address of a: " << &a << endl;
    cout << "Value at ptr: " << *ptr << endl;
    cout << "Address stored in ptr: " << ptr << endl;
    return 0;
}
```

Output:

```
Value of a: 10
Address of a: 0x7ffe5c44
Value at ptr: 10
Address stored in ptr:
0x7ffe5c44
```

3 Pointer Operators

Operator	Name	Description	Example
&	Address of	Returns address of a variable	&a
*	Dereference	Access value at the address	*ptr
->	Arrow	Access member of structure/class using pointer	ptr->name

4 Pointer to Pointer

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int a = 5;
    int *p = &a;
    int **pp = &p;
    cout << "a = " << a << endl;
    cout << "*p = " << *p << endl;
    cout << "**pp = " << **pp << endl;
```

Output:

```
a = 5
*p = 5
**pp = 5
```

5 What is a Reference?

- A reference is an alternate name for an existing variable.
- It is created using & (reference operator).
- Once initialized, a reference cannot be changed to refer to another variable.
- It must be initialized at the time of declaration.
- References are safer and easier to use than pointers for general cases.

6 Declaration and Initialization

Syntax:

```
data_type &ref_name = variable_name;
```

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int a = 10;
    int &ref = a;

    cout << "Value of a: " << a << endl;
    cout << "Value of ref: " << ref << endl;
    ref = 20;
    cout << "After modification: " << a << endl;
    return 0;
}
```

Output:

```
Value of a: 10
Value of ref: 10
After modification: 20
```

7 Pointers vs References

Feature	Pointer	Reference
Symbol	* (asterisk)	& (ampersand)
Meaning	Stores address of a variable	Alternate name for a variable
Initialization	Can be initialized later	Must be initialized during declaration
Can be NULL	Yes	No
Can be changed to refer another variable	Yes	No
Syntax	int *ptr;	int &ref;
Usage	Dynamic memory, arrays, etc.	Function arguments, safer access

8 Key Points

- Pointers store memory addresses, references are alternate names.
- Use * to access value at address, & to get address.
- Reference must be initialized and cannot be changed.
- Pointers can be NULL, references cannot.
- References are often preferred in function parameters.

Dynamic Memory + new / delete

1 What is Dynamic Memory?

- Memory that is allocated at runtime (during program execution).
- In C++, we use **new** operator to allocate memory and delete operator to deallocate memory.
- Dynamic memory allows flexible memory management (size can be decided at runtime).
- Useful when we don't know the exact size in advance.

2 new Operator

- Allocates memory in heap at runtime.
- Returns a pointer to the allocated memory.
- Syntax varies for single variable and arrays.

Syntax:

```
data_type *ptr = new data_type;    // single variable
data_type *ptr = new data_type[size];  // array
```

3 delete Operator

- Frees the dynamically allocated memory.
- Use delete for single variable and delete[] for arrays.
- Helps to avoid memory leaks.

Syntax:

```
delete ptr;      // single variable
delete[] ptr;    // array
```

4 Example: Single Variable

Example:

```
#include <iostream>
using namespace std;
int main()
{
    int *p = new int;     // allocate memory
    *p = 25;              // store value
    cout << "Value: " << *p << endl;
    delete p;             // free memory
    p = nullptr;          // avoid dangling pointer
    return 0;
}
```

Output:

```
Value: 25
```

5 Example: Dynamic Array

Example:

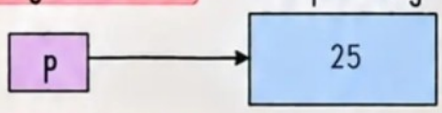
```
#include <iostream>
using namespace std;
int main()
{
    int n;
    cout << "Enter size: ";
    cin >> n;

    int *arr = new int[n];   // allocate array

    for(int i = 0; i < n; i++)
    {
        cout << "Enter element " << i << ": ";
        cin >> arr[i];
    }

    cout << "Elements: ";
    for(int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;

    delete[] arr;          // free array memory
    arr = nullptr;
    return 0;
}
```

Output:

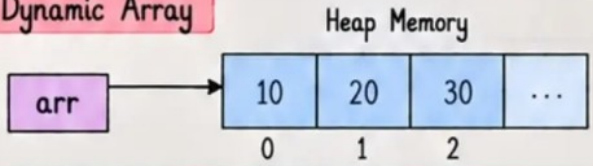
```
Enter size: 3
Enter element 0: 10
Enter element 1: 20
Enter element 2: 30
Elements: 10 20 30
```

6 Memory Representation

Single Variable



Dynamic Array



Memory allocated using **new** is stored in Heap and is accessible using a pointer.

7 Key Points

- Use **new** to allocate memory at runtime.
- Use **delete** (or **delete[]**) to free memory.
- Always use **delete** for memory allocated with **new**.
- Use **delete[]** for arrays.
- Set pointer to **nullptr** after deletion.
- Avoid memory leaks (memory not freed).
- Dynamic memory is useful for linked lists, trees, graphs, large arrays, etc.

OOP Introduction + Class & Object

1 What is OOP?

- OOP (Object Oriented Programming) is a programming paradigm based on the concept of objects.
- It helps to organize code in a more real-world and modular way.
- C++ supports OOP features like classes, objects, encapsulation, inheritance, polymorphism and abstraction.

2 Advantages of OOP

- Modularity - Code is divided into classes and objects.
- Reusability - Code can be reused.
- Maintainability - Easy to modify and manage.
- Security - Data can be hidden using access specifiers.
- Real-world modeling - Helps to model real-world problems easily.

3 Class in C++

- A class is a user-defined data type that acts as a blueprint for creating objects.
- It contains data members (variables) and member functions (methods).
- It does not occupy memory.

Syntax:

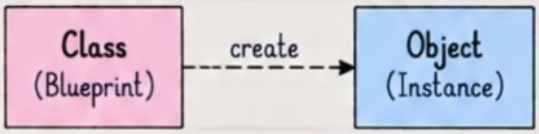
```
class ClassName
{
    // access specifier
    data members;
    member functions;
};
```

4 Object in C++

- An object is an instance of a class.
- It is created from a class.
- Each object has its own copy of data members.
- Objects occupy memory.

Syntax:

```
ClassName objectName;
Example: Student s1;
```



5 Example: Class and Object

Example:

```
#include <iostream>
using namespace std;
class Student
{
    public:
        int id;
        string name;
        void display()
        {
            cout << "ID: " << id << endl;
            cout << "Name: " << name << endl;
        }
};
int main()
{
    Student s1;        // object of class Student
    s1.id = 101;
    s1.name = "Narayan";
    s1.display();
    return 0;
}
```

Output:

```
ID: 101
Name: Narayan
```

6 Accessing Members using Dot Operator

- We use dot operator (.) to access data members and member functions of an object.
- Syntax: objectName.memberName

Example:

```
Student s1;
s1.id = 101;      // access data member
s1.name = "Narayan";
s1.display();     // call member function
```

7 Class vs Object

Class	Object
Blueprint or template	Instance of a class
Does not occupy memory	Occupies memory
Used to define data and functions	Used to access data and functions
One class can create multiple objects	Each object has its own copy of data members

8 Key Points

- Class is a blueprint, object is an instance.
- Use dot operator to access members.
- A class can have multiple objects.
- Objects of the same class can have different values.
- OOP makes code modular, reusable and easy to maintain.

OOP Introduction + Class & Object

1 What is OOP?

- OOP (Object Oriented Programming) is a programming paradigm based on the concept of objects.
- It helps to organize code in a more real-world and modular way.
- C++ supports OOP features like classes, objects, encapsulation, inheritance, polymorphism and abstraction.
- It makes the program easy to understand, modify and maintain.

2 Basic Terminologies

Term	Meaning
Class	A user-defined data type (blueprint).
Object	Instance of a class.
Data Members	Variables inside a class.
Member Functions	Functions inside a class.
Encapsulation	Binding of data and functions together.
Abstraction	Hiding unnecessary details.
Inheritance	Reusing properties of existing class.
Polymorphism	Same interface, different behavior.

3 Class Syntax

Syntax:

```
class ClassName
{
    // access specifier
    data members;
    member functions;
};
```

4 Example: Class Definition

Example:

```
#include <iostream>
using namespace std;
class Student
{
    public:
        int id;
        string name;
        void display()
        {
            cout << "ID: " << id << endl;
            cout << "Name: " << name << endl;
        }
};
```

5 What is an Object?

- An object is an instance of a class.
- It is created from a class.
- Each object has its own copy of data members.
- Objects occupy memory.
- Different objects can have different values.

6 Creating and Using Object

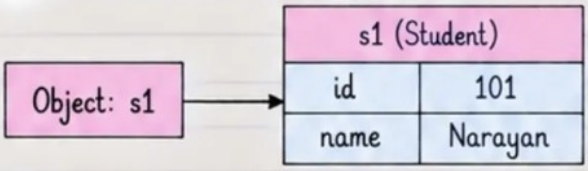
Example:

```
#include <iostream>
using namespace std;
class Student
{
  public:
      int id;
      string name;
      void display()
      {
          cout << "ID: " << id << endl;
          cout << "Name: " << name << endl;
      }
};
int main()
{
    Student s1;          // creating object
    s1.id = 101;
    s1.name = "Narayan";
    s1.display();
    return 0;
}
```

Output:

```
ID: 101
Name: Narayan
```

7 Memory Representation



8 Key Points

- Class is a blueprint, object is an instance.
- Class does not occupy memory, object does.
- An object can access data members and member functions.
- Multiple objects can be created from the same class.
- Each object has its own copy of data members.
- OOP provides modularity, reusability and easy maintenance.

Encapsulation + Access Specifiers

1 What is Encapsulation?

- Encapsulation means wrapping data (variables) and methods (functions) inside a single unit (class).
- It hides the internal details and provides controlled access to the data.
- It helps in data security and prevents direct modification of data.
- Achieved using access specifiers (private, public, protected).

2 Advantages of Encapsulation

- Data hiding and security.
- Controlled access to data.
- Prevents accidental modification.
- Improves code maintainability.
- Helps in achieving modularity.

3 Access Specifiers in C++

- Access specifiers define the visibility of class members (variables and functions).
- C++ provides three access specifiers:

private	public	protected

4 Access Specifiers Table

Access Specifier	Within Class	Within Same File (Non-class)	In Derived Class	Outside Class (Other File)
private	Yes	No	No	No
public	Yes	Yes	Yes	Yes
protected	Yes	No	Yes	No

5 Syntax

```
class ClassName
{
    private:
        // private members
    public:
        // public members
    protected:
        // protected members
};
```

6 Example: Encapsulation

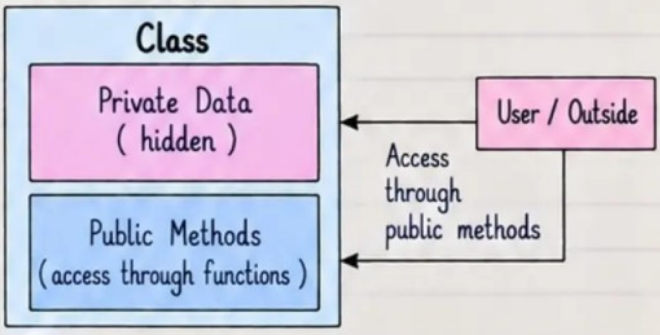
Example:

```
#include <iostream>
using namespace std;
class BankAccount
{
    private:
        int balance;    // private data member
    public:
        void setBalance(int b)
        {
            if(b >= 0)
                balance = b;
        }
        int getBalance()
        {
            return balance;
        }
};
int main()
{
    BankAccount acc;
    acc.setBalance(1000);
    cout << "Balance: " << acc.getBalance();
    return 0;
}
```

Output:

```
Balance: 1000
```

Diagram: Encapsulation



7 Key Points

- Encapsulation is implemented using access specifiers.
- private members cannot be accessed outside the class.
- public members can be accessed from anywhere.
- protected members can be accessed in derived classes.
- Encapsulation helps in data hiding, security and better code management.

Inheritance + Types of Inheritance

1 What is Inheritance?

- Inheritance is an OOP feature that allows a new class (derived class) to acquire the properties and methods of an existing class (base class).
- It helps in code reusability and establishes a relationship between classes.
- The existing class is called Base Class (Parent Class) and the new class is called Derived Class (Child Class).

2 Advantages of Inheritance

- Code reusability.
- Reduces code duplication.
- Easy to extend and maintain.
- Helps in achieving hierarchical design.
- Supports the concept of real-world relationships (is-a relationship).

3 Syntax

```
class BaseClass
{
      // data members and methods
};
class DerivedClass : access_specifier BaseClass
{
      // own members
};
```

4 Example: Single Inheritance

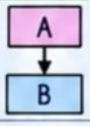
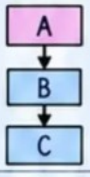
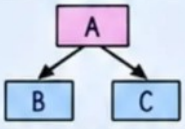
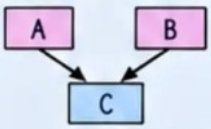
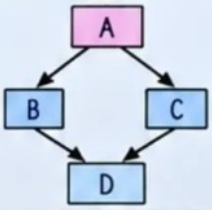
Example:

```
#include <iostream>
using namespace std;
class Animal
{
   public:
      void eat()
      {
          cout << "Animal eats" << endl;
      }
};
class Dog : public Animal
{
   public:
      void bark()
      {
          cout << "Dog barks" << endl;
      }
};
int main()
{
      Dog d;
      d.eat();    // inherited method
      d.bark();   // own method
      return 0;
}
```

Output:

```
Animal eats
Dog barks
```

5 Types of Inheritance

Type	Diagram	Description
Single Inheritance		One derived class inherits from one base class.
Multilevel Inheritance		A class inherits from a derived class.
Hierarchical Inheritance		Multiple derived classes inherit from one base class.
Multiple Inheritance		A class inherits from multiple base classes.
Hybrid Inheritance		Combination of two or more types of inheritance.

6 Key Points

- Inheritance represents an is-a relationship.
- Base class members can be inherited by the derived class.
- Use access specifiers (public, protected, private) to control the visibility.
- C++ supports single, multilevel, hierarchical, multiple and hybrid inheritance.
- Inheritance helps in code reusability and easy maintenance.
- Constructor and Destructor are not inherited, but they can be called using derived class objects.

Polymorphism & Virtual Functions

1 What is Polymorphism?

- Polymorphism means "many forms".
- It allows the same function name to behave differently for different objects.
- Provides a single interface with multiple implementations.
- Makes the code flexible, extensible and easy to maintain.

2 Types of Polymorphism

- Compile-time Polymorphism (Static Binding)
  - Function Overloading
  - Operator Overloading
- Run-time Polymorphism (Dynamic Binding)
  - Function Overriding (using virtual functions)

3 Function Overloading (Compile-time)

- Same function name with different parameters (number, type or order).
- Occurs at compile time (static binding).
- Example: same function name print() with different parameter types.

Example:

```
#include <iostream>
using namespace std;
class Math
{
   public:
      void print(int a)
      {   cout << "Integer: " << a << endl; }
      void print(double b)
      {   cout << "Double: " << b << endl; }
      void print(string s)
      {   cout << "String: " << s << endl; }
};
int main()
{
    Math m;
    m.print(10);      // integer version
    m.print(3.14);    // double version
    m.print("C++");   // string version
    return 0;
}
```

Output:

```
Integer: 10
Double: 3.14
String: C++
```

4 Function Overriding (Run-time)

- Same function name and same parameters in base and derived class.
- Occurs at run time (dynamic binding).
- Achieved using virtual functions.
- Used to achieve run-time polymorphism.

5 Virtual Functions

- A virtual function is a member function in base class that is declared with the keyword virtual.
- It allows the derived class to provide its own implementation.
- When called through a base class pointer/reference, the derived class function is executed (run-time binding).

Example:

```
#include <iostream>
using namespace std;
class Animal
{
   public:
      virtual void sound()  // virtual function
      {   cout << "Animal Sound" << endl; }
};
class Dog : public Animal
{
   public:
      void sound() override   // function overriding
      {   cout << "Dog Barks" << endl; }
};
int main()
{
    Animal *a;
    Dog d;
    a = &d;          // base class pointer
    a->sound();      // calls derived class function
    return 0;
}
```

Output:

```
Dog Barks
```

6 Compile-time vs Run-time Polymorphism

Feature	Compile-time Polymorphism	Run-time Polymorphism
Also called	Static Binding	Dynamic Binding
Achieved by	Function Overloading Operator Overloading	Function Overriding (using virtual)
Binding time	Compile time	Run time
Flexibility	Less	More
Example	print(int), print(double)	virtual void sound()

7 Key Points

- Polymorphism allows the same interface to work in different ways.
- Function overloading is a compile-time polymorphism.
- Function overriding (using virtual) is a run-time polymorphism.
- Virtual functions are used for dynamic binding.
- Run-time polymorphism helps in achieving flexibility and extensibility in large projects.

Abstraction + Abstract Class

1 What is Abstraction?

- Abstraction means hiding the implementation details and showing only the essential features (what to do, not how to do).
- It focuses on the interface rather than the implementation.
- It helps in reducing complexity and increasing security.
- Achieved using abstract classes and interfaces.

2 Advantages of Abstraction

- Hides implementation details.
- Reduces complexity.
- Improves code security.
- Helps in focusing on essential features.
- Makes code easier to maintain.

3 Abstract Class in C++

- An abstract class is a class that contains at least one pure virtual function.
- It cannot be instantiated (we cannot create its object).
- It is used as a base class and must be inherited.
- Derived class must implement all pure virtual functions.

4 Syntax of Abstract Class

```
class ClassName
{
    public:
        virtual returnType functionName() = 0;
        // other members
};
```

5 Example: Abstract Class

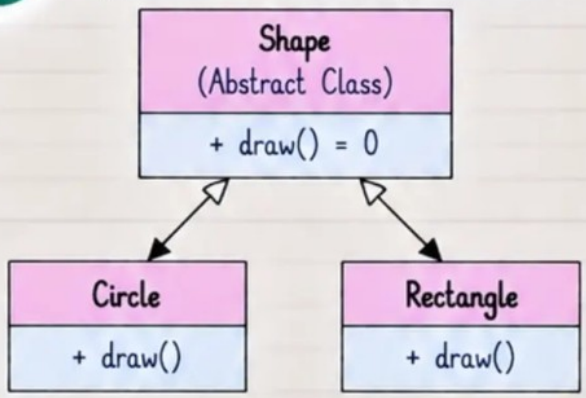
Example:

```
#include <iostream>
using namespace std;
class Shape
{
    public:
        virtual void draw() = 0;    // pure virtual
};
class Circle : public Shape
{
    public:
        void draw() override
        {
            cout << "Drawing Circle" << endl;
        }
};
int main()
{
    // Shape s;    // Error: cannot create object
    Circle c;
    c.draw();
    return 0;
}
```

Output:

```
Drawing Circle
```

6 Diagram: Abstraction using Abstract Class



7 Key Points

- Abstraction hides the internal implementation details.
- Abstract class contains at least one pure virtual function.
- We cannot create an object of an abstract class.
- Derived class must implement all pure virtual functions.
- Abstract class can have both abstract and non-abstract (normal) functions.
- Abstraction can also be achieved using interfaces (in other languages).

STL Basics (vector, stack, queue, map)

1 What is STL?

- STL (Standard Template Library) is a powerful library in C++.
- It provides ready-to-use data structures, algorithms and utilities.
- It is a part of C++ Standard Library.
- Helps in writing efficient and clean code.
- Located inside the <iostream>, <vector>, <stack>, <queue>, <map> etc. headers.

2 Components of STL

Component	Description
Containers	Store data (e.g. vector, stack, queue, map)
Algorithms	Provide functions (e.g. sort, find, etc.)
Iterators	Used to traverse containers
Functors	Function objects
Adapters	Provide different interfaces (e.g. stack, queue)

3 Vector

- vector is a dynamic array (resizable array).
- Elements are stored in contiguous memory.
- Provides fast access using index.
- Header: #include <vector>

Example:

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    for(int x : v)
        cout << x << " ";
    return 0;
}
```

Output:

```
10 20 30
```

Common Functions	
v.push_back(x)	Add element
v.pop_back()	Remove last
v.size()	Return size
v[i]	Access element
v.clear()	Remove all

4 Stack

- stack follows LIFO (Last In First Out).
- Elements can be inserted and removed only from the top.
- Header: #include <stack>

Example:

```
#include <iostream>
#include <stack>
using namespace std;
int main()
{
    stack<int> s;
    s.push(10);
    s.push(20);
    s.push(30);
    cout << s.top() << endl;
    s.pop();
    cout << s.top() << endl;
    return 0;
}
```

Output:

```
30
20
```

Common Functions	
s.push(x)	Insert element
s.pop()	Remove top
s.top()	Access top
s.empty()	Check empty
s.size()	Return size

5 Queue

- queue follows FIFO (First In First Out).
- Insertion is done at rear and deletion from front.
- Header: #include <queue>

Example:

```
#include <iostream>
#include <queue>
using namespace std;
int main()
{
    queue<int> q;
    q.push(10);
    q.push(20);
    q.push(30);
    cout << q.front() << endl;
    q.pop();
    cout << q.front() << endl;
    return 0;
}
```

Output:

```
10
20
```

Common Functions	
q.push(x)	Insert element
q.pop()	Remove front
q.front()	Access front
q.back()	Access last
q.empty()	Check empty
q.size()	Return size

6 Map

- map stores key-value pairs in sorted order of keys.
- Each key is unique.
- Internally implemented using balanced BST (usually Red-Black Tree).
- Header: #include <map>

Example:

```
#include <iostream>
#include <map>
using namespace std;
int main()
{
    map<int, string> m;
    m[1] = "One";
    m[2] = "Two";
    m[3] = "Three";
    cout << m[2] << endl;
    m[2] = "TWO";
    cout << m[2] << endl;
    return 0;
}
```

Output:

```
Two
TWO
```

Common Functions	
m[key] = value	Insert/Update
m.find(key)	Find key
m.erase(key)	Remove key
m.size()	Return size
m.empty()	Check empty
m.begin(), m.end()	Iterators

7 Key Points

- queue => FIFO

Exception Handling + File Handling

1 What is Exception Handling?

- Exception handling is a mechanism to handle runtime errors (exceptions) in a program.
- It prevents abnormal termination of the program.
- It allows us to catch and handle errors using try, catch and throw.
- Helps in writing robust and reliable code.

2 Keywords

Keyword	Purpose
try	Code that may throw an exception
catch	Handles the exception
throw	Used to explicitly throw an exception
throws	Declares that a function may throw an exception

3 Syntax

```
try
{
    // risky code
}
catch (ExceptionType e)
{
    // handle exception
}
```

Multiple catch blocks can be used for different exception types.

4 Example: Exception Handling

```
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    try
    {
        if(b == 0)
            throw "Division by zero!";
        cout << "Result: " << a / b << endl;
    }
    catch (const char* msg)
    {
        cout << "Exception: " << msg << endl;
    return 0;
}
```

Output:

```
Enter two numbers:
10 0
Exception:
Division by zero!
```

5 Key Points

- Exceptions are runtime errors.
- Use try-catch to handle exceptions.
- throw is used to raise an exception.
- Multiple catch blocks can be used.
- Helps in writing safe and reliable programs.

6 What is File Handling?

- File handling allows read, write and manipulate data in files.
- Files are stored on secondary storage.
- C++ provides fstream library for file handling.
- Used to store data permanently.

7 File Classes (in <fstream>)

Class	Purpose
ifstream	For reading from a file (input)
ofstream	For writing to a file (output)
fstream	For both reading and writing

8 Common File Modes

Mode	Meaning
ios::in	Open for reading
ios::out	Open for writing
ios::app	Append (write at end)
ios::ate	Open and move to end
ios::trunc	Delete existing content
ios::binary	Open in binary mode

9 Example: Write and Read a File

Write to a File (ofstream)

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ofstream fout("test.txt");
    if(!fout)
    {
        cout << "Cannot open file";
        return 0;
    }
    fout << "Hello C++!" << endl;
    fout << "File Handling Demo";
    fout.close();
    cout << "Data written
        successfully!";
    return 0;
}
```

Read from a File (ifstream)

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    ifstream fin("test.txt");
    if(!fin)
    {
        cout << "Cannot open file";
        return 0;
    }
    string line;
    cout << "File Content:" << endl;
    while(getline(fin, line))
    {
        cout << line << endl;
    }
    fin.close();
    return 0;
}
```

10 Key Points

- Use <fstream> library for file handling.
- Use ofstream to write and ifstream to read.
- Always close the file after operation.
- Check if the file opened successfully.
- Files help in storing data permanently.